Components of the Spark API Stack

This content will explore the four main components of the Spark API stack, helping you understand the differences and their roles in data processing.

Learning Objective

By the end of this train, you should be able to:

- List and compare the four components of the Spark API stack.

Spark Components

> The Spark application is not just a single tool, but rather allows for the processing of big data, dealing with graph objects, performing machine learning, and streaming.

One of the distinguishing features of Spark is the unification of the processing components, having a single engine for batch, stream, machine learning, and graph processing. All of the workloads are separate from the underlying Spark engine. Thus, when writing code in one of the above APIs, it will be converted into a DAG for execution on the underlying fault-tolerant engine, being converted into a highly compact bytecode for execution on the workers' JVMs.

1. SparkSQL

This module is specifically for processing structured data (RDBMS, CSV, Parquet, Avro, JSON, ORC, etc.). The API first reads in the data from one of the aforementioned sources and converts it to a temporary or permanent Spark table.
This can then be manipulated using one of the programming language APIs, or using SQL queries directly. Spark Structured APIs are ANSI SQL:2003-compliant and can be used as a pure SQL engine as well.

We can actually convert a table into a temporary view:

```
# Create data to read into the DataFrame.
data = [(1, 'doge', 23),
        (2, 'pepe', 44),
        (3, 'steve', 65),
        (4, 'brian', 0)]

columns = ['id', 'name', 'count']

# Read the data as a DataFrame using the createDataFrame method.
df = spark.createDataFrame(data, columns)

# Create a SparkSQL view using the createOrReplaceTempView() method.
df.createOrReplaceTempView("temp_df")
```

Let's run through what we have just done:

1. Created a list of tuples, which will each form a row within a DataFrame we want to create.
2. Created a list of column names for the DataFrame.
3. Created a DataFrame by using the createDataFrame() method of the SparkSession and specifying the data generated in 1, and field names in 2.
4. Created a view from the DataFrame.

Now we can use the SparkSQL API to query the view we created:

```
# The spark.sql() method returns a DataFrame on which we can call the show()
method to print the results

spark.sql("SELECT * FROM temp_df").show()
```

You can see that we use the `sql()` method to run SQL queries on the underlying views or tables. You will also start to become familiar with another method: `show()` to visualize the data.

2. Spark MLlib

This module contains common machine learning algorithms, built on top of the Spark DataFrame-based APIs. This API allows users to extract and transform features, build pipelines for model training and prediction, and persistence models during deployment. In addition to basic machine learning algorithms, this API contains common linear algebra operations and statistics, as well as lower-level ML primitives, such as a gradient descent optimizer. This API also closely resembles the .fit .transform paradigm first implemented in the scikit-learn python package.

3. Spark Structured Streaming

This module is built atop the SparkSQL engine and DataFrame-based APIs. The Stream API views an incoming stream as a continually growing table, with new data being added as rows to the end of the table. This allows developers to treat the table in the same way as they would a static table. This module also accepts a wide range of sources, including Apache Kafka, Kinesis, and HDFS-based or cloud storage.

4. Spark GraphX

This module is primarily for the creation and manipulation of graph objects, performing graph-parallel operations. GraphX allows standard graph operations. A lot of these functions are contributed by the Spark community.

Comparison of the Four Components

Component	Data Type	Processing Model	Key Strengths
Spark SQL	Structured/Semi-structured	Batch & Interactive	SQL support, optimizations
Spark Streaming	Real-time streams	Micro-batch (Structured Streaming) Continuous mode optional	Low-latency, fault-tolerant
Spark MLlib	Tabular (DataFrames)	Batch (Training/Inference)	Scalable ML pipelines
Spark GraphX	Graph data (Nodes & Edges)	Batch (Graph ops)	Optimized for graph algorithms

Basic Spark Concepts

When using Spark, it is important to understand some of the basic concepts in the Spark application and how the code is transformed and executed as tasks across the Spark executors.

- **Application**

  The complete Spark program as written by a user. Consists of a driver program and executors on the cluster.

- **SparkSession**

  programmatic object that provides a point of entry to interaction with an underlying Spark engine and allows programming Spark through its APIs. In the shell (also in

Databricks) this is instantiated for you, but you have to set this up if coding up a Spark application.

- **Job**

  A parallel computation that consists of multiple tasks and is created in response to a Spark action (`collect`, `show`, `write`). Spark transforms this into a DAG, which is Spark's execution plan. Each node within a DAG can be single or multiple Spark stages.

- **Stage**

  Each job is divided into smaller tasks, called stages, that depend on each other.

- **Task**

  A single unit of work that is sent to the Spark executor.

Conclusion

In conclusion, the Spark API stack provides a comprehensive suite of tools for diverse big data processing needs, with Spark SQL handling structured data queries, Spark Streaming enabling real-time analytics, MLlib supporting scalable machine learning, and GraphX specializing in graph-based computations. Each component is optimized for its specific use case while leveraging Spark's core distributed engine, ensuring efficiency, scalability, and seamless integration across batch and real-time workflows. Together, they empower users to tackle complex data challenges, from ETL and predictive modeling to graph analysis and streaming, within a unified framework.

▼Key Takeaways ✖

- **Takeaway 1:** Spark SQL, best for structured queries (SQL/DataFrames).
- **Takeaway 2:** Spark Streaming, best for real-time data processing.
- **Takeaway 3:** Spark MLlib, best for scalable machine learning.
- **Takeaway 4:** Spark GraphX, best for graph-based computations.
- **Takeaway 5:** All components, run on top of Spark's core engine (RDDs) and share the same execution model.